Assignment 1 - Two Clocks: A Bitemporal Football League

Trifan Alexandru - MariaDB 11.8.9

1. Domain and why time matters

The database models one football league in the 2025/26 season: 70 players, 6 clubs, the contracts and loans between them, each club's head coach and each club's budget. It receives 23 changes, from the go-live on 1 July 2025 to May 2026.

Football paperwork is often late and sometimes wrong. So every fact has two dates: when it was true (valid time) and when the database knew it (transaction time).

Example: who was Harbor City's head coach on 4 March 2026? Asked on 10 March, the database says Romano. On 16 March a clerk records that Kelly took over on 2 March, so on 20 March the answer is Kelly. On 23 March the date is corrected to 6 March, so today the answer is Romano again.

All three answers are right for their moment. If the league fined the coach for the 4 March match, the fine issued on 20 March can only be explained with the 20 March view, while an appeal needs today's view. One clock cannot give both.

2. Modeling decisions

Two clocks. valid_from / valid_to are DATE, because football changes by the day; tx_from / tx_to are DATETIME(6) in UTC. Both are half-open [from, to), so Weber until 6 October and Larsen from 6 October do not overlap. Every table has CHECK (valid_from < valid_to) and CHECK (tx_from < tx_to).

Sentinels, not NULL. An open end is 9999-12-31 (valid) or 9999-12-31 23:59:59.999999 (transaction). One test, from <= x AND x < to, then works for every row and can use an index.

Anchors and versions. player, club and coach (anchors) hold only what never changes. Three version tables hold the history: contract_version (the relationship), head_coach_version (the role) and club_version (name and budget, the changing number). Foreign keys point only at anchors, because a version table has many rows per entity. club_version also gives each club's league membership, which I2 checks contracts and coaches against.

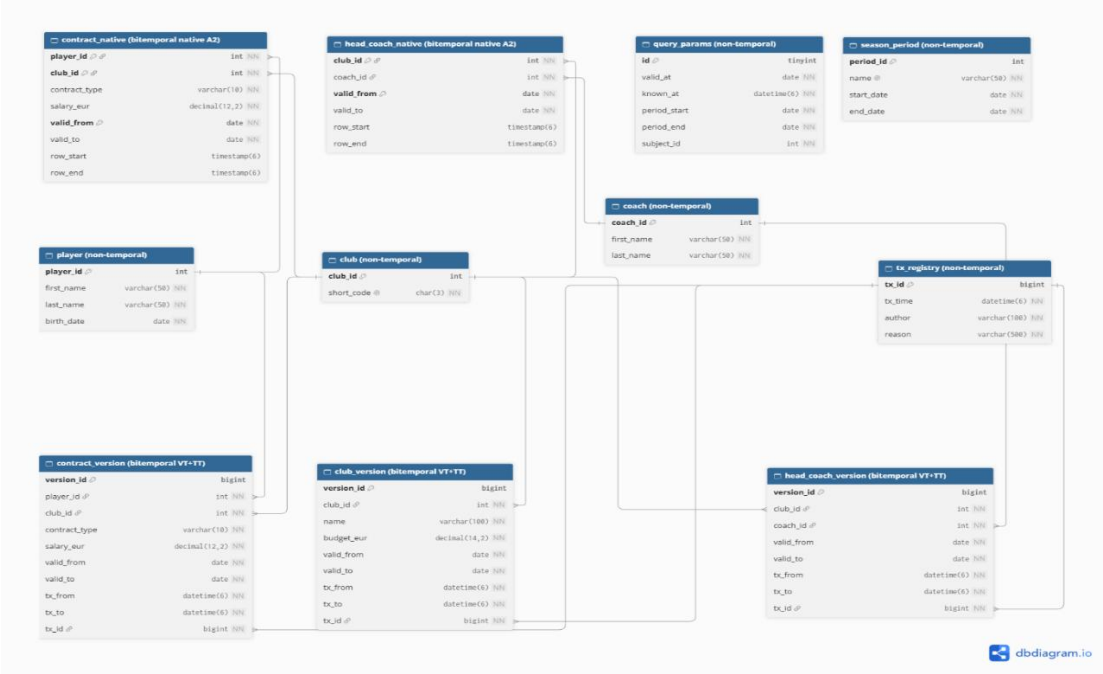
Contracts and loans. A contract's key is (player, club). A loan is a LOAN row at the new club while the PERMANENT row at the home club keeps running, so they do not clash under I1. The same pair can stop and start again (E6).

One change, one timestamp. Each change sets the simulated clock (SET @sim_now = '...'), adds one row to tx_registry(tx_id, tx_time, author, reason) and stamps all its rows with that tx_id and tx_from, even across tables (S4 changes a loan and a budget). Each reason starts with the story label, e.g. 'S5 | E3+E5: ...'.

Non-temporal tables. Anchors never change; season_period is the fixed league calendar; tx_registry is the transaction log itself and is only appended to; query_params holds the inputs for the views.

Indexes. contract_version has (player_id, tx_to, valid_from) for Q1 and (club_id, tx_to, valid_from) for Q6; head_coach_version has (club_id, tx_to, valid_from).

Simplifications. A loan does not store the lending club; it is read from the permanent contract. Matching tx_from to tx_registry.tx_time is done by the write protocol, not by a key. Anchors keep no history.



3. Write protocol

Every change: set the clock, log it in tx_registry, close the old row (tx_to = now), insert the new row. The only UPDATE ever run sets tx_to; nothing is deleted. The scripts come from 03_timeline.sql.

E3, late entry (S22). Moreau's raise at Riverside (640,000 → 720,000 from 1 March 2026) is recorded on 10 April, 40 days late. His row is closed and written again, split at 1 March.

```
START TRANSACTION;
SET @sim_now = '2026-04-10 10:00:00';
INSERT INTO tx_registry (tx_time, author, reason)
VALUES (@sim_now, 'clerk_bob', 'S22 | E3: Moreau raise at RIV from 1 Mar, recorded 40 days late');
SET @tx_id = LAST_INSERT_ID();
UPDATE contract_version SET tx_to = @sim_now
WHERE player_id = 13 AND club_id = 2
  AND tx_to = @tx_open AND valid_to = @valid_open;
INSERT INTO contract_version
  (player_id, club_id, contract_type, salary_eur, valid_from, valid_to, tx_from, tx_id)
VALUES
  (13, 2, 'PERMANENT', 640000.00, '2025-07-01', '2026-03-01', @sim_now, @tx_id),
  (13, 2, 'PERMANENT', 720000.00, '2026-03-01', @valid_open,  @sim_now, @tx_id);
COMMIT;
```

E4, correction (S9). Kowalski's salary was typed as 450,000; it is 540,000 from the start. Same period, new value. The wrong row stays, closed, so the old belief can still be shown.

```
START TRANSACTION;
SET @sim_now = '2025-10-20 10:00:00';
INSERT INTO tx_registry (tx_time, author, reason)
VALUES (@sim_now, 'clerk_ana', 'S9 | E4 correction: Kowalski salary typo 450,000 -> 540,000');
SET @tx_id = LAST_INSERT_ID();
UPDATE contract_version SET tx_to = @sim_now
WHERE player_id = 45 AND club_id = 5
  AND tx_to = @tx_open AND valid_to = @valid_open;
INSERT INTO contract_version
  (player_id, club_id, contract_type, salary_eur, valid_from, valid_to, tx_from, tx_id)
VALUES (45, 5, 'PERMANENT', 540000.00, '2025-07-01', @valid_open, @sim_now, @tx_id);
COMMIT;
```

E8, retraction (S7). A loan of Rossi to Northside, entered by mistake at 08:00, is withdrawn at 16:30. The row is closed and nothing replaces it. Its valid dates stay, because the loan never existed.

```
START TRANSACTION;
SET @sim_now = '2025-09-15 16:30:00';
INSERT INTO tx_registry (tx_time, author, reason)
VALUES (@sim_now, 'clerk_ana', 'S7 | E8 retraction: Rossi loan to NOR was never agreed, entered by mistake in S6');
SET @tx_id = LAST_INSERT_ID();
UPDATE contract_version SET tx_to = @sim_now
WHERE player_id = 56 AND club_id = 1 AND tx_to = @tx_open;
COMMIT;
```

Compare E5 (S3): when Cole left Harbor City, his row was closed and written again with valid_to = 2025-08-04, because the contract was real. In A1 the same changes are CALLs to sp_assert_fact, sp_correct_fact and sp_retract_fact, and triggers block direct writes.

4. Challenges and failed attempts

Re-running the schema failed. 01_schema.sql first dropped the tables one by one. A block pasted in the wrong place recreated head_coach_version before coach was dropped: error 1451, and errno 150 on an empty database. Fix: the script now starts with DROP DATABASE IF EXISTS and recreates everything.

The rerun hung. In DataGrip, DROP DATABASE waited for a lock held by other open tabs, each one a connection. SHOW PROCESSLIST showed five sleeping connections; KILL fixed it. Now I disconnect before a full rerun.

The index did not fit Q6. The first index started with player_id, but Q6 filters by club, so EXPLAIN would show no difference. I added (club_id, tx_to, valid_from) inside CREATE TABLE; added later, MariaDB's own foreign-key index on club_id would have hidden the difference.

“Throughout” per row was wrong. Moreau's raise on 1 March splits his contract into two rows, so neither row covers the whole spring. Q4 now merges touching rows into spells first.

5. Insights: built by hand vs native support

The core track builds everything by hand. The engines from the lectures offer parts of it natively:

Feature	This project (core)	MariaDB 11.8	PostgreSQL 18/19	SQL Server 2022	Oracle 23ai
Valid time	2 DATE columns, by hand	PERIOD FOR, FOR PORTION OF	range + WITHOUT OVERLAPS; FOR PORTION OF in 19	manual only	Temporal Validity (PERIOD FOR)
Transaction time	2 DATETIME(6) columns, by hand	WITH SYSTEM VERSIONING	none (triggers)	system-versioned tables	Flashback Data Archive
Backdating TT	yes (@sim_now)	yes (insert_history)	n/a	no	no
I1 / I3 (no overlap)	reported by Q0	WITHOUT OVERLAPS	WITHOUT OVERLAPS	no	no
I2 (parent covers child)	reported by Q0	no	temporal FK (PERIOD)	no	no
I4 (history immutable)	write protocol (+ A1 triggers)	enforced	no	enforced	enforced

What the engine would enforce. I1 and I3 (no overlap): WITHOUT OVERLAPS in MariaDB and PostgreSQL 18. I2 (parent covers child): only PostgreSQL 18's temporal foreign key. I4 (history cannot change): system versioning in MariaDB, SQL Server and Oracle's Flashback Data Archive. In our core track, I1–I3 are only reported by Q0, and I4 relies on the write protocol (and A1's triggers).

Catch 1: both clocks in one table. Closed and current rows of the same contract overlap in valid time, so a plain WITHOUT OVERLAPS key would reject them; PostgreSQL would need separate current and history tables. In MariaDB, unique keys on a system-versioned table include row_end, so only current rows are compared: A2 accepted all 108 history rows and still blocked an overlapping current contract.

Catch 2: backdating. Engines stamp transaction time with the real clock, so a normal load would date our whole timeline to one evening in October 2026.

A2 (MariaDB 11.8). contract_native and head_coach_native use PERIOD FOR valid_period with WITHOUT OVERLAPS, and WITH SYSTEM VERSIONING. The history was loaded with system_versioning_insert_history = ON, copying tx_from / tx_to into row_start / row_end. Q1, Q5 and Q6 use FOR SYSTEM_TIME AS OF; all 11 comparisons with the core views (EXCEPT both ways) returned the same rows, including Romano / Kelly / Romano. Valid time is still a WHERE condition, because MariaDB has no AS OF for it in SELECT. What we gave up:

- TIMESTAMP(6) instead of DATETIME(6) (rule 3); “still current” ends at 2106-02-07, not the 9999 sentinel.
- No tx_id, author or reason on native rows, so the E9 link between tables is lost.
- I2 is still not enforced.

A1. Four procedures (sp_assert_fact, sp_end_fact, sp_correct_fact, sp_retract_fact) do all writes, and triggers on the three version tables block 13 forbidden operations. The procedures set a session flag that the

triggers check. It is a convention, not security: real protection would give the application only EXECUTE rights.

The as-of index: EXPLAIN of Q6

Q6 filters contract_version on club_id, tx_to and valid_from. Below is its contract part for Harbor City on 4 March, known on 20 March, with and without the index.



id	selec…	table	type	possible_ke…	key	key_l…	ref	rows
1	1 SIMPLE	qp	ALL	<null>	<null>	<null>	<null>	1
2	1 SIMPLE	cv	ref	ix_contract_club_asof	ix_contract_club_asof	4	football_bt.qp.subject_id	16

Output — Q6.g EXPLAIN without index: ALL

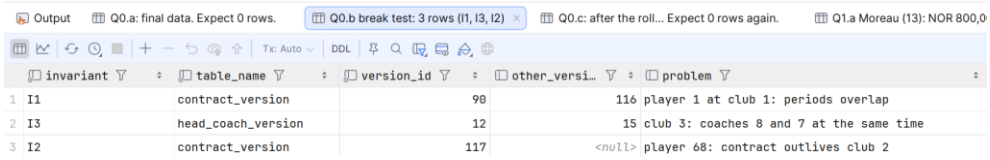
id	selec…	table	type	possible_ke…	key	key_l…	ref	rows
1	1 SIMPLE	qp	ALL	<null>	<null>	<null>	<null>	1
2	1 SIMPLE	cv	ALL	<null>	<null>	<null>	<null>	101

With the index: type = ref on ix_contract_club_asof, about 16 rows read. With IGNORE INDEX: type = ALL, all 101 rows scanned. The gap grows with the history: a scan reads the whole league, the index only one club.

Query results (Part C)

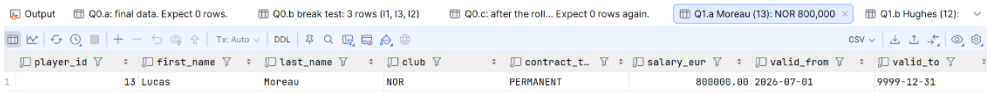
All outputs come from 05_tests.sql, run after 01–04 on an empty database.

Q0: integrity violations. Zero rows on the final data. The test adds three bad rows in a transaction (an overlapping contract, a second Harbor City coach, a contract that starts before the league exists); each rule fires once, and after ROLLBACK the view is empty again.



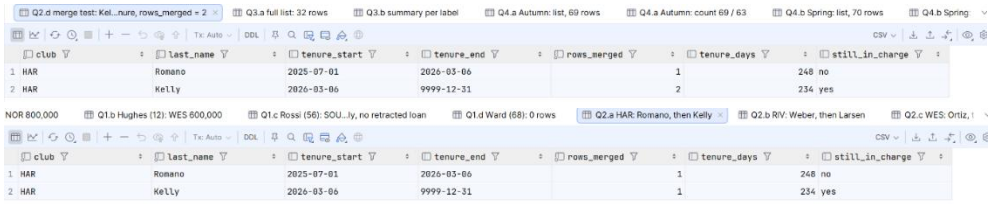
invariant	table_name	version_id	other_versi…	problem
I1	contract_version	90	116	player 1 at club 1: periods overlap
I3	head_coach_version	12	15	club 3: coaches 8 and 7 at the same time
I2	contract_version	117	<null>	player 68: contract outlives club 2

Q1: current contracts. Valid today and still believed. Moreau: only Northside, from 1 July 2026. Rossi: only Southport; the retracted loan does not appear.



player_id	first_name	last_name	club	contract_t…	salary_eur	valid_from	valid_to
13	Lucas	Moreau	NOR	PERMANENT	800000.00	2026-07-01	9999-12-31

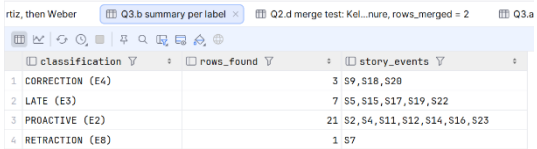
Q2: coach tenures, as known today. Harbor City: Romano until 6 March 2026 (248 days), then Kelly, using the corrected date from S20. Touching rows of the same coach are merged: the test splits Kelly's row in two and still gets one tenure (rows_merged = 2).



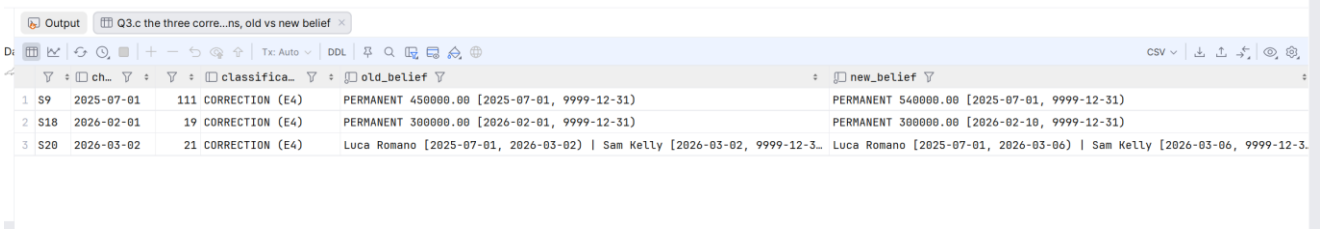
club	last_name	tenure_start	tenure_end	rows_merged	tenure_days	still_in_charge
HAR	Romano	2025-07-01	2026-03-06	1	248	no
HAR	Kelly	2026-03-06	9999-12-31	2	234	yes

club	last_name	tenure_start	tenure_end	rows_merged	tenure_days	still_in_charge
HAR	Romano	2025-07-01	2026-03-06	1	248	no
HAR	Kelly	2026-03-06	9999-12-31	1	234	yes

Q3: late, proactive and corrected changes. A change also re-writes the unchanged parts (Cole's S3 row would look 35 days late), so Q3 compares the rows a change closed with the rows it wrote and finds the first date where they differ. Correction: an old row starts on that date. Proactive: the date is after the recording day. Late: 7 or more days before it. Corrections come from the data, not from tags. The 32 rows match the story log's E2, E3 and E4 events exactly.

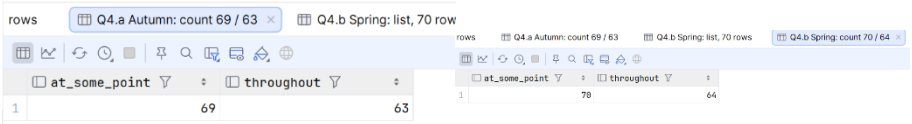


classification	rows_found	story_events
CORRECTION (E4)	3	S9,S18,S20
LATE (E3)	7	S5,S15,S17,S19,S22
PROACTIVE (E2)	21	S2,S4,S11,S12,S14,S16,S23
RETRACTION (E8)	1	S7

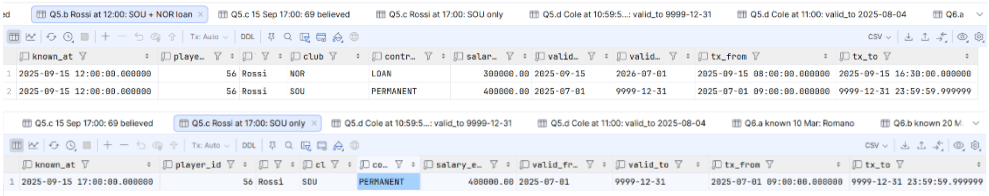


ch…		classifica…	old_belief	new_belief
S9	2025-07-01	111 CORRECTION (E4)	PERMANENT 450000.00 [2025-07-01, 9999-12-31)	PERMANENT 540000.00 [2025-07-01, 9999-12-31)
S18	2026-02-01	19 CORRECTION (E4)	PERMANENT 300000.00 [2026-02-01, 9999-12-31)	PERMANENT 300000.00 [2026-02-10, 9999-12-31)
S20	2026-03-02	21 CORRECTION (E4)	Luca Romano [2025-07-01, 2026-03-02) | Sam Kelly [2026-03-02, 9999-12-3…	Luca Romano [2025-07-01, 2026-03-06) | Sam Kelly [2026-03-06, 9999-12-3…

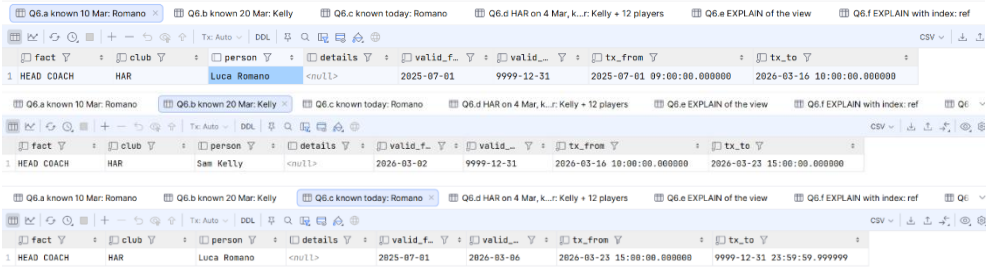
Q4: contracts in a named period. Touching rows are merged into spells first; at_some_point uses overlap and throughout uses containment, both half-open. Autumn: 69 and 63 (not throughout: Cole, Hughes, Meyer, Martin, Fischer's loan, Hughes at Westbrook). Spring: 70 and 64; Moreau counts as throughout despite his split.



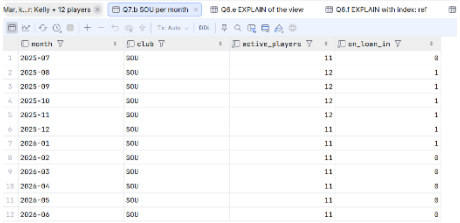
Q5: the table as believed at a moment. Only transaction time is filtered. At go-live: 66 contracts. On 15 September at 12:00: 70, including Rossi's mistaken loan. At 17:00: 69, after the 16:30 retraction, while the noon view can still be reproduced.



Q6: the three answers. Same club (Harbor City) and date (4 March 2026); only known_at changes. 10 March: Romano (go-live row). 20 March: Kelly (S19, handover 2 March). Today: Romano (S20, Romano until 6 March). The 12-player squad is the same each time.



Q7: players per club per month. A recursive CTE builds the 12 months of the season; a LEFT JOIN keeps empty months as 0. Southport: 11 in July, 12 in August–November (Fischer on loan), 11 afterwards; on_loan_in is 0 from February.



Q8: gaps before rejoining. LAG gives the previous end; gap = next start − previous end; only gaps over 0 days are shown. Exactly the three E6 events: Cole 159 days, Hughes 141 (back on loan), Meyer 72. Moreau's touching rows are correctly not a gap.

